Learning guide for Lộc: what the PA3 work did and why

Plain-language notes on the Docker, database and CI work (SCRUM-42, SCRUM-43) and the workflow around it. Every file name below is real. Where the repo does not say something, this guide says so.

Commands marked "try it" are safe. They only read, or touch your local Docker.

---

1. Docker

The analogy

- **Image** = a recipe plus frozen ingredients. Read-only. Example: `postgres:16-alpine`.
- **Container** = a dish cooked from the recipe. A running (or stopped) instance of an image.
- **Volume** = a fridge outside the kitchen. Food in it survives when you throw the dish away.
- **Healthcheck** = tasting the dish to see if it is actually ready, not just on the table.

The worked example: the `db` service in `docker-compose.yml`

```yaml
db:
  image: postgres:16-alpine            # the recipe: Postgres 16 on tiny Alpine Linux
  container_name: debatespeak-db       # the name you see in `docker ps`
  restart: unless-stopped              # come back after a crash or reboot
  environment:                         # settings handed to Postgres on first start
    POSTGRES_USER: ${POSTGRES_USER:-debatespeak}
    ...
  ports:
    - "5432:5432"                      # your-PC-port : container-port
  volumes:
    - postgres_data:/var/lib/postgresql/data   # where Postgres keeps its files
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ... -d ..."]
    interval: 5s   timeout: 5s   retries: 5   start_period: 10s
```

Line by line, the parts that confuse people:

- `${POSTGRES_USER:-debatespeak}` means: use the variable `POSTGRES_USER` (from your `.env`), and if it is not set, use `debatespeak`. So Compose works even without a `.env`.
- `ports: "5432:5432"` opens a door from your laptop to the container. Without it, only other containers could reach the database. The API running on your laptop connects to `localhost:5432`.
- `volumes: postgres_data:/var/lib/postgresql/data` mounts a named volume at the folder where Postgres stores its data. At the bottom of the file the volume is named `debatespeak_postgres_data`.
- `healthcheck` runs `pg_isready` every 5 s. It tries up to 5 times (`retries`). `start_period: 10s` gives Postgres 10 s of grace before failures count. Status goes `starting` → `healthy` (or `unhealthy`).

Why `docker compose up db` starts only the database

`docker compose up` with no names starts every service in the file (`db`, `redis`, `livekit`). If you name a service, Compose starts only that one (and anything it `depends_on`; this file has no `depends_on`, so nothing else is pulled in). PA3 only needs Postgres, so:

```
docker compose up -d --wait db
```

- `-d` = detached, runs in the background.

- `--wait` = do not return until the container is `healthy`. This is why the healthcheck matters: the command only finishes when the database really accepts connections.

This is the command SCRUM-42 added to `README.md` (PR 15).

What a "clean volume" means

Postgres only reads `POSTGRES_USER`, `POSTGRES_PASSWORD` and `POSTGRES_DB` when its data folder is **empty**. If the volume already has data from an earlier run, those settings are ignored. So "starts clean for every member" means: start from an empty volume and still reach healthy. To get an empty volume:

```
docker compose down -v      # stops containers AND deletes the volumes (your DB data is gone)
docker compose up -d --wait db
```

`-v` is the dangerous letter: it deletes the data. Fine on a dev laptop, never on anything you care about.

What SCRUM-42 actually changed

Be precise here. Comparing `origin/pa3` with `origin/feature/SCRUM-42-docker-db`, the change touches only two files: - `README.md`: adds the "PA3 only needs the database" paragraph and the `docker compose up -d --wait db` command. - `docs/pa3/loc/TODO.md`: marks the row In review and records that it was verified from a wiped volume.

The `db` service itself was **already** in `docker-compose.yml` on `pa3`. SCRUM-42 verified it and documented it. It did not edit the compose file.

Try it

```
docker compose ps                          # status of containers (look for "healthy")
docker compose logs db                     # what Postgres printed
docker exec debatespeak-db pg_isready -U debatespeak -d debatespeak
docker exec -it debatespeak-db psql -U debatespeak -d debatespeak   # a SQL prompt; \dt lists tables, \
docker volume ls                           # you should see debatespeak_postgres_data
docker compose down                        # stop, keep data
docker compose down -v                     # stop, DELETE data
```

---

2. What each service does in DebateSpeak

All three are defined in `docker-compose.yml`. Only Postgres is needed for PA3.

Postgres 16 (`db`)

The permanent memory. Stores users, debate rooms, topics, transcripts and reports. The spec (`api/app/core/database_document.md`) says it is chosen for strong relational links (a room belongs to users and a topic) plus JSONB columns for the semi-structured AI output. If the power goes out, Postgres still has the data.

Redis 7 (`redis`)

A very fast database that lives in memory. Think of a whiteboard next to the filing cabinet (Postgres). The architecture spec (`docs/architecture/SOFTWARE_ARCHITECTURE_SPEC.md`) gives it these jobs: - **Job queue.** The API puts a note on the board ("analyse this debate"), and a separate worker picks it up later. The library for this is **ARQ** (`arq` is in `api/pyproject.toml`). This keeps slow AI work out of the web request, so the page does not hang. - **Pub/sub** for transcript fan-out: one process announces "new transcript line", and the WebSocket hub sends it to both browsers. - **Rate limiting** (counters per user or IP). - Cache.

The README calls Redis the "in-memory cache and async worker queue (ARQ)". These are Build 3 and 4 features. PA3 does not use Redis yet.

Try it:

```
docker exec debatespeak-redis redis-cli ping      # expect PONG
```

LiveKit (`livekit`)

Carries the live audio between the two debaters. It is an SFU (a server that forwards each person's audio track). Browsers connect to it with WebRTC, and the spec says each speaker has their own track, so the system knows who said what. The compose file runs it with `--dev`, which means a throw-away local server with the dev key `devkey` / secret `secret` (see `.env.example`). Port 7880 is the HTTP/WebSocket door; 7881 (TCP) and 7882/udp carry the audio. - Not used in PA3 (Increment 2 / PA4, per `docs/ROADMAP.md`). - Compose uses the image tag `latest` for LiveKit, so it can change under you. That is a small reproducibility risk; pinning a version would be safer. (My observation, not something the repo discusses.)

---

3. Alembic migrations and the seed script

Migrations

A database has a **schema** (tables and columns). Code changes over time, so the schema must too. A **migration** is a small, numbered script that moves the schema one step ("create table users", "add column level"). Alembic is the Python tool that runs them. Think of Git for your database structure.

```
alembic upgrade head        # apply all migrations that have not run yet
alembic downgrade -1        # undo the last one
alembic current             # which step is this database at?
```

Why bother: every teammate and CI start with an empty database. Migrations make all of them end up with the same tables.

What is in the repo today: `api/alembic.ini` exists and `alembic` is a dependency. **There are no migration scripts and no seed script on `pa3` yet.** They are SCRUM-41 (not part of your work). Until that lands, `alembic upgrade head` has nothing real to run.

Seed script

A seed script fills a fresh database with known sample rows so everyone can test. `docs/pa3/README.md` says the goal is `python -m app.seed` giving 5 users, 10 topics (all CEFR levels) and 1 room in `LOBBY`. That script does not exist on `pa3` yet either.

The intended order

```
docker compose up -d --wait db   # an empty, healthy database
alembic upgrade head             # create the tables      (from api/)
python -m app.seed               # insert sample rows
```

---

4. CI (continuous integration)

What it is

CI = a robot that, on every push and pull request, checks out your code on a clean machine and runs checks. If any step fails, the PR shows a red X. It catches "works on my laptop" problems. It is defined in `.github/workflows/ci.yml` and runs on GitHub Actions.

Walk through `.github/workflows/ci.yml`

Triggers

```
on:
  push:          # runs when you push to: main, dev, pa*, feature/**, bugfix/**
  pull_request:  # and on PRs targeting: main, dev, pa*
```

`pa*` matches `pa3`, `pa4` and so on. So your `feature/...` branch runs CI on every push, and again as a PR into `pa3`.

Concurrency

```yaml
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

If you push twice quickly on the same branch, the older run is cancelled. Saves time.

Job 1: `security-and-invariants` (runs on `ubuntu-latest`, a fresh Linux VM) 1. `actions/checkout@v4` downloads your code. Without it the VM is empty. 2. "Verify No Tracked .env Secrets": `git ls-files` lists tracked files; `grep` finds any named `.env` or `.env.something`; `grep -v '\.env\.example'` lets `.env.example` through. If anything is left, print an error and `exit 1` (= fail). This enforces the AGENTS.md rule: never commit secrets. 3. "Verify Zero Hardcoded Absolute Paths": `git grep` searches `api/`, `workers/`, `web/` for `D:\...`, `C:\Users\...` or `/home/...`. Any hit fails the job. Enforces the "no absolute paths" rule.

Job 2: `backend-ci` (`needs: security-and-invariants`, so it waits for job 1 and is skipped if job 1 fails) 1. checkout. 2. `actions/setup-python@v5` installs Python 3.11 and caches pip downloads. 3. Install: upgrade pip, install `ruff pytest pytest-asyncio httpx`, then `pip install -e "api/[dev]"` (installs the API package in editable mode plus its dev extras from `pyproject.toml`). 4. `ruff check api/ workers/ tests/`: the linter (finds bugs and unused imports). 5. `ruff format --check ...`: checks formatting without changing files; fails if code is not formatted. 6. `pytest tests/ -v`: runs the tests. The `env:` block sets `DATABASE_URL: sqlite+aiosqlite:///:memory:` (a throw-away in-memory SQLite, **not Postgres**) and `REDIS_URL`. No Redis or Postgres service is started in this job.

Job 3: `frontend-ci` (also `needs: security-and-invariants`; runs in parallel with backend) 1. checkout. 2. `actions/setup-node@v4` with Node 20 and an npm cache keyed on `web/package-lock.json`. 3. `npm ci` in `web/`: a clean, exact install from the lock file. 4. `npm run build`: TypeScript type check and Vite build.

Things worth knowing (honest observations)

- Tests run on SQLite, so CI today never proves the code works on real Postgres. That is the gap SCRUM-43 is meant to close.
- `ci.yml` sets `APP_ENV`, which matches `api/app/config.py`. `.env.example` uses `ENV` instead, which `config.py` does not read. Also the default DB password in `config.py` (`debatespeak_dev_pwd`) differs from the compose and `.env.example` one (`debatespeak_dev_password`). If your `.env` is missing, the API would try the wrong password. Not verified beyond reading the files; worth a look by the backend owner.
- I could not run the workflow or Docker here. This section is from reading the file.

SCRUM-43

A CI job that starts Postgres, runs migrations and the seed. **Status: in progress, gated until SCRUM-41 lands** (there is nothing to migrate or seed before then). It is being written separately; nothing about it is described here beyond that.

---

5. The PA3 workflow (from `docs/pa3/README.md` and `CLAUDE.md`)

`pa3 -branch-> feature/SCRUM-42-docker-db -PR-> pa3 -(end of sprint, Lộc)-> dev`

1. `git checkout pa3 && git pull`, then branch from `pa3`.
2. Name it `feature/SCRUM-<nn>-<short-name>` (bug fixes: `bugfix/SCRUM-<nn>-<short-name>`). The Jira number links the code to the task. The name also matches the CI trigger `feature/**`.
3. Open the PR **into `pa3`**, never `dev` or `main`. One teammate reviews. A merged PR counts as reviewed.
4. Nobody commits directly to `pa3`, `dev` or `main`. Why: these are the shared, always-working lines (AGENTS.md section 6: they must stay releasable). A branch lets CI and a reviewer look at a change before it can break everyone. The README's one exception is that Lộc commits planning docs straight to `pa3`; `CLAUDE.md` is stricter and tells AI agents not to. For agents, use a branch + PR.
5. Lộc merges `pa3` into `dev` once, at the end of the sprint.
6. Jira: use In Review while the PR is open. Done only after it is merged.

Review of PRs (your SCRUM-38 and 35 task) does not need a branch of its own.

---

6. Glossary

Term	Meaning
Image	Read-only template a container starts from
Container	A running instance of an image
Volume	Storage that outlives the container
Docker Compose	Tool that starts several containers from one `docker-compose.yml`
Healthcheck	A command Docker runs to decide if the container is ready
Port mapping	`host:container`, a door from your PC into the container
Postgres	The main relational database
Redis	In-memory store: cache, queue, pub/sub
ARQ	Python job-queue library that uses Redis
LiveKit / SFU	Server that forwards live audio between participants
Schema	The tables and columns of a database
Migration	A versioned script that changes the schema
Alembic	The tool that runs migrations
Seed	Script inserting known sample data
CI	Robot that runs checks on every push and PR
Job / step	A CI machine run / one command inside it
Lint	Automatic code-quality scan (ruff)
PR	Pull request: a proposal to merge a branch

---

7. Self-check

1. You run `docker compose up db`. Which containers start, and why not the others?
2. You changed `POSTGRES_PASSWORD` in `.env`, but the old password still works. Why, and what command fixes it?
3. What does `--wait` add, and what in the compose file makes it possible?
4. Today's CI tests use SQLite. Name one thing that means CI cannot catch.
5. Why do you branch and open a PR instead of committing to `pa3`?

Answers 1. Only `db` (Postgres). Compose starts just the services you name plus their `depends_on` ones; there are none here. 2. Postgres reads the password only when the data folder is empty. The existing volume keeps the old one. `docker compose down -v` then `docker compose up -d --wait db` (this deletes the data). 3. It waits until the container is `healthy` before returning. The `healthcheck` block (`pg_isready`) makes it possible. 4. Real Postgres behaviour: for example Postgres-only types or SQL, and whether Alembic migrations and the seed work against it. 5. `pa3` must stay working for everyone. A branch plus PR lets CI and a reviewer check the change before it merges.